Brt Dci Contracts

Prodigy

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Oct 10, 2024 - Oct 14, 2024

SOURCE CODE
brt-dci-contracts

ASSESSED COMMIT ID
24946ea

SEVERITY BREAKDOWN

11	11	0	0
All Findings	Solved	Risk Accepted	Acknowledged



Introduction

`Prodigy` engaged Halborn to conduct a security assessment on their smart contracts revisions beginning on *10/10/2024* and ending on *10/14/2024*. The security assessment was scoped to the smart contracts provided to the Halborn team.

Assessment Summary

The team at Halborn was provided 3 days for the engagement and assigned a full-time security engineer to evaluate the security of the smart contract.
The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.
The purpose of this assessment is to:

- Ensure that smart contract functions operate as intended.
- Identify potential security issues with the smart contracts.

In summary, Halborn identified some security risks that were addressed by the `Prodigy team`.

Test Approach and Methodology

Halborn performed a combination of manual and automated security testing to balance efficiency, timeliness, practicality, and accuracy regarding the scope of this assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance code coverage and quickly identify items that do not follow the security best practices. The following phases and associated tools were used during the assessment:

- Research into architecture and purpose.
- Smart contract manual code review and walkthrough.
- Graphing out functionality and contract logic/connectivity/functions. (`solgraph,draw.io`)
- Manual assessment of use and safety for the critical Solidity variables and functions in scope to identify any arithmetic related vulnerability classes.
- Manual testing by custom scripts.
- Static Analysis of security for scoped contract, and imported functions. (`Slither`)
- Testnet deployment. (`Hardhat`, `Foundry`)

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY

(a) Repository: brt-dci-contracts

(b) Assessed Commit ID: 24946ea09d682ebb64075f0c5abcfc3b33b4c39b

(c) Items in scope:

- AggregatorHelper
- DCIStructs
- Factory
- PythAggregator
- Router
- TransferHelper
- Vault
- VaultKeeper

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Price Manipulation Vulnerability in Vault Execution Due to Unchecked Pyth Oracle Updates	High	8.6	SOLVED 10/28/2024
HAL-002	Chainlink Oracle Price Feed Used Without Staleness Check	Medium	6.3	SOLVED 10/28/2024
HAL-003	Excess ETH Not Refunded in Price Update Transactions	Medium	6.3	SOLVED 10/28/2024
HAL-004	Zero Amount Transfer Vulnerability in Token Transfers	Medium	6.3	SOLVED 10/28/2024

#	TITLE	SEVERITY	SCORE	STATUS
HAL-005	Pyth oracle price is not validated properly	Medium	4.7	SOLVED 10/28/2024
HAL-006	Unrestricted Vault Creation in Factory Contract	Low	3.1	SOLVED 10/28/2024
HAL-007	Unsafe Casting Operations	Low	3.1	SOLVED 10/28/2024
HAL-008	Incorrect Fee Calculation Due to Delayed Initialization in Vault Contract	Low	3.1	SOLVED 10/28/2024
HAL-009	Incorrect State Modification Order in lpWithdraw Function	Low	3.1	SOLVED 10/28/2024
HAL-010	Missing Visibility Attribute	Informational	—	SOLVED 10/28/2024
HAL-011	Consider Using Named Mappings	Informational	—	SOLVED 10/28/2024

Findings & Tech Details

HAL-001 SOLVED

Price Manipulation Vulnerability in Vault Execution Due to Unchecked Pyth Oracle Updates

High · 8.6 AO:A/AC:L/AX:L/C:N/I:L/A:N/D:M/Y:M/R:N/S:C

DESCRIPTION

The `execute` function in Vault.sol allows arbitrary `priceUpdateData` to be passed, which is used to fetch the oracle price:

SOL

CODE REDACTED

This `priceUpdateData` is passed through `AggregatorHelper.sol` and then to `PythAggregator.sol` :

SOL

CODE REDACTED

The issue is that there's no validation of `_priceUpdateData`. A malicious user can provide empty price data, causing the `updatePrice` function to skip the update entirely.
A vault executor could then skip or avoid the updating of the Pyth price by passing in an empty `priceUpdateData` array.
Link to Base Pyth UpdatePriceFeeds

SOL

CODE REDACTED

CODE REDACTED

This leads to the `getEmaPrice` function potentially returning an outdated price, which is then used to determine the vault's state.
Ref 1
Ref 2

PROOF OF CONCEPT

Actors:

- Alice: An honest user who created a vault
- Bob: A malicious user looking to exploit the system

Initial Setup:

1. Alice creates a "Buy Low" vault with the following parameters:
   - Linked Price: 1000 USDC per ETH
   - Expiry: 7 days from now
2. The current market price of ETH is 1100 USDC.

Exploit Scenario:

1. 7 days pass, and the vault is ready for execution.
2. The current market price of ETH has dropped to 990 USDC.
3. Bob, being a malicious actor, notices this price drop and sees an opportunity to exploit the system.
4. Bob calls the `execute` function on Alice's vault, but instead of providing the current price data, he provides an empty `priceUpdateData` array.
5. The `execute` function passes this empty array through the system:
   - Vault.sol calls AggregatorHelper.sol
   - AggregatorHelper.sol calls PythAggregator.sol

- PythAggregator.sol's `updatePrice` function doesn't perform any update due to the empty array
- PythAggregator.sol's `getEmaPrice` function returns the last stored price, which is 1010 USDC

6. The `execute` function uses this outdated price of 1010 USDC to determine the vault's state.
7. Since 1010 USDC is greater than the Linked Price of 1000 USDC, the vault's state is set to "unswapped" (state = 2).
8. As a result, Alice's "Buy Low" strategy fails to execute, even though the actual market price (990 USDC) is below her Linked Price.

Outcome:

- Alice loses the opportunity to buy ETH at her desired price, even though market conditions were favorable.
- Bob potentially benefits if he's an LP or has opposing positions.

RECOMMENDATION

It is recommended to implement a strict validation of the `priceUpdateData` in the `execute` function of Vault.sol, mostly when `execute()` is called with Pyth Oracle:

SOL

CODE REDACTED

A modifier can also be implemented to always verify this.

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/Vault.sol#L121

REMEDIATION COMMENT

SOLVED: A modifier has been implemented to handle 0 length `priceUpdateData` array :

SOL

CODE REDACTED

HAL-002 SOLVED

Chainlink Oracle Price Feed Used Without Staleness Check

Medium · 6.3 AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:M/R:N/S:C

DESCRIPTION

The `getPrice` function in AggregatorHelper.sol fetches price data from a Chainlink oracle without validating the freshness of the returned data:

SOL

CODE REDACTED

The function calls `latestRoundData()` on the Chainlink oracle but does not check the `updatedAt` timestamp to ensure the price data is current. This allows stale or outdated price information to be used in critical calculations.

RECOMMENDATION

It is recommended to implement a staleness check using the `updatedAt` value returned by `latestRoundData()`. Compare it against the current block timestamp and a predefined threshold:

SOL

CODE REDACTED

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/AggregatorHelper.sol#L30

REMEDIATION COMMENT

SOLVED: Checks have been implemented to ensure price returned by chainlink is correct and not stale.

HAL-003 SOLVED

Excess ETH Not Refunded in Price Update Transactions

Medium · 6.3 AO:A/AC:L/AX:L/C:N/I:N/A:N/D:M/Y:N/R:N/S:C

DESCRIPTION

The `PythAggregator` contract's `updatePrice` function, which is called indirectly through `getPrice()`, does not refund excess ETH sent by users. This function is called in multiple payable functions across `Router.sol`, `Factory.sol`, and `Vault.sol`, allowing users to provide `msg.value`. The relevant code in `PythAggregator.sol` is:

SOL

CODE REDACTED

This implementation transfers the exact fee to the `Pyth` contract but does not refund any excess ETH sent by the user. The excess ETH remains trapped in the `PythAggregator` contract.
Users who inadvertently send more ETH than required for price updates will lose their excess funds. This results in a direct financial loss for users and accumulation of unusable ETH in the `PythAggregator` contract. The severity of this issue is heightened by the fact that it affects multiple core functions of the protocol in Router.sol,Factory.sol and of course Vault.sol

RECOMMENDATION

It is recommended to implement a refund mechanism in the `updatePrice` function to return any excess ETH to the user. Here's an example of how to modify the function:

SOL

CODE REDACTED

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/PythAggregator.sol#L42

REMEDIATION COMMENT

SOLVED: Excess fees are now refunded to `msg.sender.`

HAL-004 SOLVED

Zero Amount Transfer Vulnerability in Token Transfers

Medium · 6.3 AO:A/AC:L/AX:L/C:N/I:N/A:M/D:N/Y:N/R:N/S:C

DESCRIPTION

The contracts contain multiple instances call of `safeTransfer()` and `safeTransferFrom()` calls that do not check if the transfer amount is greater than zero. This issue is present in the following locations:

- In the init() function:

SOL

CODE REDACTED

- In the deposit() function:

SOL

CODE REDACTED

- In the withdraw() function:

SOL

CODE REDACTED

- In the _lpWithdrawInvestmentToken() function:

SOL

CODE REDACTED

- In the _lpWithdrawLinkedToken() function:

SOL

CODE REDACTED

- In the lpWithdraw() function:

SOL

CODE REDACTED

CODE REDACTED

- In the lpCancel() function:

SOL

CODE REDACTED

These transfers are executed without verifying if the amount is greater than zero. Some ERC20 tokens, such as older implementations or non-standard tokens, revert when a transfer amount of zero is attempted. The absence of a zero amount check can cause these transfers to fail and revert the entire transaction.

Refs :

- Solodit-Caviar
- Solodit-Concur

RECOMMENDATION

To address this, it is recommended to implement zero-amount checks before all token transfers. Add a condition to skip the transfer if the amount is zero. Here's an example of how to modify the code:

SOL

CODE REDACTED

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/Vault.sol#L169

REMEDIATION COMMENT

SOLVED: Checks for non 0 amounts have been added to the needed functions.

HAL-005 SOLVED

Pyth oracle price is not validated properly

Medium · 4.7 AO:A/AC:L/AX:L/C:N/I:L/A:L/D:L/Y:N/R:N/S:C

DESCRIPTION

The `PythAggregator` contract does not perform input validation on the `price`, `conf`, and `expo` values returned from the called price feed, which can lead to the contract accepting invalid or untrusted prices. Those values should be checked as clearly stated in the official documentation.

RECOMMENDATION

It is recommended that the contract revert the transaction here if one of the following conditions is triggered:

- `price <= 0`
- `expo < -18`
- `conf > 0 && (price / int64(conf) < MIN_CONF_RATIO` for a given `MIN_CONF_RATIO`

REFERENCES

https://solodit.xyz/issues/m-01-pyth-oracle-price-is-not-validated-properly-pashov-audit-group-none-nabla-markdown

REMEDIATION COMMENT

SOLVED: Pyth price now have checks about `price`, `expo` and `conf.`

HAL-006 SOLVED

Unrestricted Vault Creation in Factory Contract

Low · 3.1 AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:N/S:C

DESCRIPTION

The Factory.sol contract allows unrestricted vault creation, contradicting the documented requirement for Liquidity Providers (LPs). The createVault() function lacks access control:

SOL

CODE REDACTED

This implementation permits any address to create vaults without authorization checks.

RECOMMENDATION

It is recommended to either modify the documentation or implement strict access control for vault creation.

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/Factory.sol#L178

REMEDIATION COMMENT

SOLVED: Documentation has been updated.

HAL-007 SOLVED

Unsafe Casting Operations

Low · 3.1 AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:N/S:C

DESCRIPTION

In the AggregatorHelper.sol contract, several unsafe casting operations are performed without using `SafeCast` library. This could potentially lead to unexpected overflows or underflows in certain edge cases.
The following unsafe casting operations were identified:

1. `int32(-1 * int8(IChainlinkOracle(_aggregator).decimals()))`
2. `uint256(uint64(price)) * 10 ** uint32(targetDecimals - priceDecimals)`
3. `uint256(uint64(price)) / 10 ** uint32(priceDecimals - targetDecimals)`

These operations involve casting between different integer types (`int8`, `int32`, `uint64`, `uint256`) without using SafeCast library to prevent potential overflows or underflows.
Unsafe casting could potentially lead to unexpected behavior or vulnerabilities in edge cases or future modifications of the contract.

RECOMMENDATION

It is recommended to consider using OpenZeppelin's `SafeCast` library for all casting operations to prevent potential overflows or underflows. This is a best practice that enhances the overall robustness and security of the contract.

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/AggregatorHelper.sol#L57

REMEDIATION COMMENT

SOLVED: SafeCast is now used everywhere.

HAL-008 SOLVED

Incorrect Fee Calculation Due to Delayed Initialization in Vault Contract

Low · 3.1 AO:A/AC:L/AX:L/C:N/I:N/A:N/D:L/Y:N/R:N/S:C

DESCRIPTION

The Vault contract calculates fees during the `init()` function call instead of at contract creation. This delay allows for a discrepancy between the price at contract creation and the price at initialization.

Relevant code snippets:

SOL

CODE REDACTED

This leads to incorrect fee calculations. A malicious vault creator could exploit this by:

1. Deploying the contract when prices are high.
2. Waiting for prices to drop.
3. Calling `init()` when prices are low, resulting in lower fees.

RECOMMENDATION

It is recommended to calculate and store the fees in the constructor to ensure they are based on the price at contract creation.

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/Vault.sol#L79

REMEDIATION COMMENT

SOLVED: Fees are still computed in constructor but added to `linkedTokenAmount` or `investmentTokenAmount` so the problem is solved.

HAL-009 SOLVED

Incorrect State Modification Order in lpWithdraw Function

Low · 3.1 AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:N/S:C

DESCRIPTION

The `lpWithdraw` function in the Vault contract violates the Checks-Effects-Interactions (CEI) pattern. The `lpWithdrawn` state variable is set to true after external interactions are performed, allowing for potential reentrancy attacks. The problematic code is as follows:

SOL

CODE REDACTED

RECOMMENDATION

To mitigate this, it is recommended to implement the Checks-Effects-Interactions pattern by updating the `lpWithdrawn` state variable before performing any external calls. Restructure the function as follows:

SOL

CODE REDACTED

CODE REDACTED

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/Vault.sol#L216

REMEDIATION COMMENT

SOLVED: `lpWithdrawn = true;` is now set right after the require statement.

HAL-010 SOLVED

Missing Visibility Attribute

Informational · AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:N/S:C

DESCRIPTION

It is best practice to set the visibility of state variables and constants explicitly.

SOL

CODE REDACTED

RECOMMENDATION

Consider explicitly setting the visibility of all state variables and constants.

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/Vault.sol#L36C5-L37C30

REMEDIATION COMMENT

SOLVED: Each variable has now a visibility set.

HAL-011 SOLVED

Consider Using Named Mappings

Informational · AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:N/S:C

DESCRIPTION

Using named mappings can improve the readability and maintainability of the code by making the purpose of each mapping clearer. This practice helps developers and auditors understand the mappings' intent more easily.

SOL

CODE REDACTED

RECOMMENDATION

Consider refactoring the mappings to use named arguments, which will enhance code readability and make the purpose of each mapping more explicit.

SOL

CODE REDACTED

REFERENCES

https://github.com/prodigyfi/brt-dci-contracts/blob/master/src/Vault.sol#L41

REMEDIATION COMMENT

SOLVED: Mappings are now using named arguments.

Automated Testing

Halborn used automated testing techniques to enhance the coverage of certain areas of the smart contracts in scope. Among the tools used was `Slither`, a Solidity static analysis framework. After Halborn verified the smart contracts in the repository and was able to compile them correctly into their abis and binary format, Slither was run against the contracts. This tool can statically verify mathematical relationships between Solidity variables to detect invalid or inconsistent usage of the contracts' APIs across the entire code-base.

Slither-1.png

Slither-2.png

All issues identified by `Slither` were proved to be false positives or have been added to the issue list in this report.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.